LEARNING GUIDE + PRESENTATION KIT

AccessWindow: a content-filtering proxy with an approval workflow

Written for someone who has never seen this project. Read sections 1–6 to understand it, section 9 to build your talk, section 10 to survive the questions.

---

1. The idea in plain English

> Every connection goes through a **middleman (a proxy)**. The proxy checks an **ordered list of rules**. The **first rule that matches wins**. A web **dashboard** lets an admin edit the rules and approve people's requests for **temporary access**, which **expires by itself**.

Analogy: the school security desk.
Students don't walk straight into other buildings. They go via a desk. The guard has a clipboard of rules, top to bottom: *"Bob may enter the Lab until 3pm" | "Nobody enters the Lab"*. The guard reads from the top and obeys the **first line that applies**. If Bob asks for access, the principal (admin) writes a new line **at the top** with an expiry time. When 3pm passes the line stops applying, and the old "Nobody enters" line takes over again. The guard never needs to be told to remove it.

What problem does it solve?

- Block unwanted sites (e.g. YouTube) for some or all users, in one central place the user can't switch off from the browser.
- Let people ask for a short exception, with a reason, and let an admin approve it, without editing config files.
- Inspect and filter content (keywords, card numbers, executable downloads) in a lab environment.
- Keep an audit log of who tried to reach what.

The two things you will say most often

"First match wins."
Rules are checked top to bottom. Order is the logic. That's why a temporary *allow* is inserted at position 0.

"Expiry is enforced in the data path."
The proxy itself compares `expires_at` with the clock on every request, so even if the dashboard crashed, access would still end on time.

---

2. Vocabulary you need

Proxy / forward proxy
A server that makes requests on behalf of a client. In a *forward* proxy the client is configured to use it (this project). A *reverse* proxy sits in front of servers instead (load balancers, CDNs).

ACL (access control list) / rule
An ordered list of allow/deny conditions. Here: who (user), what (domain, service, path, method), when (schedule), from where (IP range).

Default action
What happens if no rule matches. *Default deny* = whitelist (safer). *Default allow* = blacklist (practical for browsing).

HTTP `CONNECT`
How a browser asks a proxy for an HTTPS tunnel: `CONNECT youtube.com:443`. The proxy then just copies encrypted bytes back and forth. It can only see the **hostname**, not the page or content.

TLS / HTTPS
Encryption between browser and website.

Mutual TLS (mTLS)
Both sides show certificates. The client certificate's Common Name (e.g. `bob`) becomes the user's identity, which can't be faked by editing a header.

CA (certificate authority)
The thing that signs certificates so others trust them. `gen_certs.py` makes a project CA.

TLS interception
The proxy pretends to be the website using its own CA-signed certificate. Needed here only to show a block page for HTTPS sites.

Immutable object
Never modified after creation. Policy changes build a *new* Policy and swap it in, so threads never see half-edited rules.

CSRF

A malicious site tricks your logged-in browser into submitting a form to the dashboard. Defence: a secret token in every form.

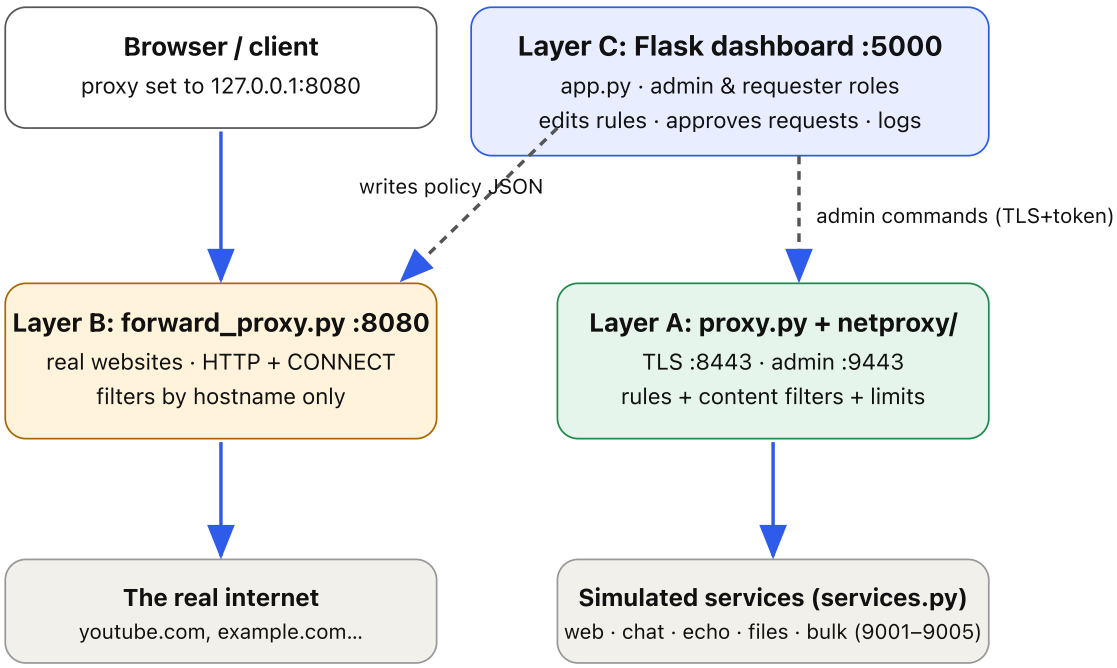
SSRF
Tricking a server into fetching internal addresses (localhost, LAN). Relevant to the Browse page.

Service worker
A script a site installs in your browser that can serve cached pages offline. It's why a blocked YouTube first looked "offline".

---

3. Architecture: three layers

The project is really three programs that cooperate. Think **two proxies** (data plane) and **one dashboard** (control plane).



Layer	Filters what?	Policy file	Default
A netproxy (original academic proxy)	Connections to *simulated* services, and the **content** of the stream	`config/policy.json`	deny
B forward_proxy.py	**Real** domains, by hostname	`config/real_website_policy.json`	allow
C app.py (Flask)	Nothing directly. It is the **control plane**.	edits both	–

Why two proxies? A is a full TLS proxy for a closed lab with identities and content inspection. B is a lightweight proxy so a normal browser can be filtered on the real web.

Which file does what (the 10 you should recognise)

File	One-line job
`forward_proxy.py`	Layer B: reads `CONNECT`, authenticates, runs `match_rule()`, tunnels or blocks
`blockpage.py`	Local CA + per-host certs to show a real "blocked" page over HTTPS
`app.py`	Dashboard: login, roles, requests, approvals, Browse, website rules
`netproxy/policy.py`	The rule engine: immutable `Policy`, `evaluate()`, atomic swap, file watcher
`netproxy/server.py`	Layer A server: thread per client, TLS, relay and inspect
`netproxy/filters.py`	Keyword/regex/MIME inspection and the stream re-assembler
`netproxy/httpmsg.py`	Strict HTTP parser (anti request-smuggling)
`netproxy/registry.py`	Live connection table, limits, access log, `revalidate()`
`netproxy/control.py`	Admin console (BLOCK, RULE, KILL, STATUS…)
`services.py`	The five fake back-end servers used for the lab

Roughly 9,000 lines of Python in total. The only third-party dependency for the core is the standard library; the dashboard needs Flask.

---

4. The story: one complete user journey

This is the heart of your demo. Learn it as a story with four acts.

ACT 1: BLOCKED

1. Admin adds a rule on the **Website access** page: `deny * → youtube.com`.
2. Bob opens youtube.com. The browser sends `CONNECT youtube.com:443` to the proxy on :8080.
3. The proxy checks Bob's proxy password (`407` if wrong), then runs `match_rule("youtube.com","bob")`. The deny rule matches → **verdict: deny**. It logs the event and never opens a connection to YouTube.
4. With `--block-page` Bob sees a real page: why it is blocked, by whom, and a **Request temporary access** button.

ACT 2: REQUEST

1. Bob signs into the dashboard and submits: site = youtube.com, 5 minutes, a reason.
2. The app only accepts sites the proxy *currently blocks for Bob* (it uses the same `match_rule` function, so the two can't disagree). Nothing changes yet. The request is **pending**.

ACT 3: APPROVE

1. Admin opens **Access requests** and clicks **Approve**. Only admins can; it's enforced on the server (`@role_required("admin")`), not just by hiding a button.
2. A rule is inserted at **index 0**:

```
{"action":"allow","users":["bob"],"domains":["youtube.com"],
 "temporary":true,"expires_at": now + 5*60, "grant_reason":"…"}
```

3. Bob reloads: first match is now this *allow* → the proxy opens a tunnel. Carol is still blocked, because the rule names only Bob.

ACT 4: EXPIRE OR REVOKE

1. Five minutes pass. `rule_expired()` returns true, the proxy **skips** that rule, and the older deny matches again. No cleanup job is needed for correctness.
2. Already-open tunnels are not forgotten: every second `tunnel()` re-runs `match_rule` and stops at the deadline. An admin **Revoke** cuts a live YouTube stream within about a second.
3. `purge_expired_real_rules()` later tidies dead rules out of the JSON file (cosmetic only).

> Design decision to defend: **where is expiry enforced?** In the proxy's data path, not just in a background job. Failure of the dashboard can't leave a site open.

---

5. Rule engine: try it yourself

This simulator mirrors `match_rule()` from `forward_proxy.py`. Change the user, the site and whether the temporary grant is still valid, then watch which rule wins.

User bob | Site youtube.com | Bob's 5-minute grant still valid

#	Rule	Action	Users	Domain	Result
1	REAL-GRANT temporary, 5 min	allow	bob	youtube.com	**MATCH, first match wins**
2	REAL-DENY admin block	deny	*	youtube.com	not reached
–	default	allow	*	*	not reached

ALLOW: proxy opens a tunnel

Decided by REAL-GRANT.

Things to notice

- `www.youtube.com` and `googlevideo.com` are covered by the `youtube.com` rule (subdomain match plus the `DOMAIN_FAMILIES` table).
- `notyoutube.com` is **not** blocked. The match is `host == d or host.endswith("." + d)`, so a look-alike suffix doesn't trick it. There's a test for this.
- Default action is **allow** in Layer B. In Layer A it is **deny**.

The actual code (it's short)

```
def match_rule(host, user):
    host = host.lower().rstrip(".")
    policy = read_policy()
    for rule in policy.get("rules", []):
        if not rule.get("enabled", True) or rule_expired(rule): continue
        users = rule.get("users", ["*"])
        if users and "*" not in users and user not in users: continue
        if domain_hit(host, rule):
            return rule.get("action", "deny"), rule      # FIRST MATCH WINS
    return policy.get("default_action", "allow"), None
```

Layer A's richer rules

Layer A rules can also match user groups (`@staff`), source network, service, **path** (`/admin/*`), HTTP method and a **time schedule**. The shipped sample policy:

ID	Effect
R01	Staff allow
R02	Quarantined network deny
R03	Anonymous users can't use chat/files
R04	Deny `/admin` on web
R05	Deny `/social` Mon–Fri 09:00–17:00
R06	Students allow
R07	Guests: web + echo only

Two-stage evaluation: at *connect* time the proxy knows user, IP, service and time. At *request* time it also knows method and path. A path rule can't be decided at connect time, so a deny is deferred and an allow admits provisionally.

Path normalisation (`normalize_path`) turns `/%61dmin`, `//admin` and `/x/../admin` into `/admin` *before* matching, and the proxy forwards the normalised path, so what was checked is exactly what is served.

Dynamic policy: `Policy` is immutable. Any change builds version+1 and swaps it atomically. A watcher polls the file each second; invalid files are rejected and the old policy stays. After each swap, `revalidate()` re-checks every open connection and kills the ones now denied.

---

6. Content filtering (Layer A)

keyword
words, optionally whole-word, case-insensitive.
F01 gambling, F03 "confidential", F05 profanity, F06 "password".

regex
patterns.
F02 card numbers → `[CARD REDACTED]`.

mime
the response `Content-Type`.
F04 blocks Windows executables (staff exempt).

Action	Effect
BLOCK	Request refused (`403`) or, on a stream, the session is terminated
REDACT	Matched text replaced (e.g. `***`), traffic continues
LOG	Recorded only (audit)

If several match: **any block wins outright → then redactions → log filters only record.**

The clever part: keywords split across packets

TCP is a stream with no message boundaries. One `recv()` can return half a line. If you only scan each chunk, `"confi"` + `"dential"` slips through. `StreamInspector` buffers bytes into complete lines and holds back any tail that could be the start of a blocked word. A test (`test_every_segmentation_gives_same_result`) checks the verdict is identical however the data is chopped.

Why strip `Accept-Encoding`? A gzip body can't be scanned, so the proxy asks the server for uncompressed responses.

7. Security design (good talking points)

Threat	Defence
Rule bypass via odd URLs	`normalize_path()`; forward the normalised path
HTTP request smuggling	`httpmsg` rejects Content-Length + Transfer-Encoding together, conflicting lengths, etc.
Impersonating a user	Identity from the TLS client certificate, not a client-supplied header
Keyword split across packets	`StreamInspector` line re-assembly + hold-back
Look-alike domain (`notyoutube.com`)	exact or `"." + domain` suffix match
Requester approves own request	server-side `@role_required("admin")`
CSRF	per-session token on every POST, compared with `hmac.compare_digest`
Password theft from the data store	PBKDF2-SHA256 salted hashes (never plaintext)
SSRF through the Browse page	http/https only; host must resolve to a **public** IP; every redirect re-checked; login required
Scripts in a fetched preview	sandboxed iframe, no `allow-scripts`, no `allow-same-origin`
Rogue certificates from the block-page CA	dedicated CA, key `chmod 600` and git-ignored, 30-day leaf certs, strict host-name validation, only blocked hosts intercepted, trust removed after the demo
Resource exhaustion	connection semaphore, per-user caps, sliding-window rate limit, size limits, timeouts
Bad policy edit	validated; invalid → rejected, old version kept

Why a browser can't just show "blocked"

For HTTPS the browser sends `CONNECT` *before* TLS. If the proxy refuses, browsers throw away the refusal's body and show their own error. To show a real page, the proxy must answer `200` and do the TLS handshake itself with a certificate from its own CA, which the browser must trust once. This is called **TLS interception**; it's only done for **blocked** hosts. Allowed traffic stays end-to-end encrypted.

8. Honest limitations

Presenters who name their own limits sound credible. Say these before someone asks.

1. **Hostname-only for HTTPS.** It can block `youtube.com` but not one video on it.
2. **Bypassable.** Only traffic sent *to* the proxy is filtered. A VPN, DNS-over-HTTPS or switching the proxy off avoids it. Real enforcement needs a gateway or firewall.
3. **Demo-grade authentication.** Layer B uses Basic proxy auth (cleartext on the wire) with public demo passwords and a non-constant-time `==` compare. Fine on loopback only.
4. **JSON file as the database.** Writes are atomic but two admins editing at once could race.
5. **Layer B detects revocation by polling** every second; Layer A is pushed the change instantly.
6. **Flask development server**, not production.
7. **Layer A listens on 0.0.0.0** and ships with a default admin token and no mandatory client certificate. Tighten before any shared use.

Next steps you could propose: TLS interception with a managed CA (to see paths), a real database, hashed proxy credentials, push notifications, SSO, audit export.

Test status: 86 tests, 85 pass. The one failure (`test_tls_and_identity`) expects TLS 1.3 but this Mac's system Python negotiates TLS 1.2. It's an environment difference, not a logic bug. Run with `venv/bin/python -m unittest discover -s tests` .

---

9. Your presentation: a 10-minute plan

Eight slides. Timings add up to ~10 minutes with demo. "Say" lines are suggested wording; rephrase in your own voice.

> **Slide 1: Title and problem** (0:45)
>
> *"Schools and offices need to control which sites people can reach, but a hard block frustrates people with a genuine need. AccessWindow is a filtering proxy that blocks by default and lets users request temporary access that an admin approves and that expires on its own."*

> **Slide 2: What is a proxy?** (1:00)
>
> Diagram: Browser → Proxy → Website. Mention forward vs reverse, `CONNECT` for HTTPS.
>
> *"The browser never talks to the website directly. The proxy decides first. For HTTPS it can only see the hostname, not the page."*

> **Slide 3: Architecture** (1:15)
>
> Use the diagram from section 3. Three layers: two proxies and one dashboard.
>
> *"Layer A is the lab proxy with identity and content inspection. Layer B filters real websites by hostname. Layer C, the dashboard, is the control plane that edits the rules."*

> **Slide 4: The rule engine** (1:30)
>
> Show the 8-line `match_rule`. Example list: allow bob→youtube (temporary), deny *→youtube, default allow.
>
> *"Rules are checked top to bottom and the first match wins. So a temporary allow goes on top and beats the deny below it. When it expires, the deny applies again."*

> **Slide 5: The approval workflow** (1:00)
>
> Four-step strip: Blocked → Request → Approve → Expire.
>
> *"Expiry is checked by the proxy on every request, not by a cleanup job, so it still works if the dashboard crashes. Open tunnels are re-checked every second, so a revoke cuts a live video within about a second."*

> **Slide 6: LIVE DEMO** (2:30)
>
> Follow section 11. **Have a backup:** screenshots or `packet_flow.html` in case the network or proxy fails.

> **Slide 7: Security and limitations** (1:15)
>
> Pick 3 defences (first-match path normalisation, CSRF + server-side roles, SSRF guard) and 2 limits (hostname-only HTTPS, bypassable).
>
> *"It's a demo-grade system and I know exactly where it stops: it can't see inside HTTPS, and anyone who turns the proxy off bypasses it."*

> **Slide 8: Summary and questions** (0:45)
>
> *"Ordered rules, first match wins, temporary access is just a rule with an expiry, enforced in the data path. Happy to take questions."*

Delivery tips

- Memorise the **one-sentence summary** from section 1 and the **four-act story** from section 4. Everything else is detail you can look up.
- If you don't know an answer, say what the code *does* and where you'd look (`policy.py`, `forward_proxy.py`) instead of guessing.
- Rehearse the demo twice, including the failure case (what you do if the proxy won't start).

10. Likely questions and short answers

What does your project do?

It filters network access through a proxy using ordered rules. A dashboard lets an admin manage rules and approve requests for temporary access, which expires automatically.

Why use a proxy instead of filtering in the browser?

It's central, outside the user's control, works for any app that honours the proxy, and gives one place to log and audit.

Can the proxy see which page I visit on an HTTPS site?

No. Only the hostname from `CONNECT`. The path and content are inside TLS. Seeing them would need TLS interception.

What does "first match wins" mean and why does order matter?

Rules are tested top to bottom and the first whose conditions all match decides. A temporary allow must sit above the deny to override it, so it's inserted at position 0.

How does a site re-block itself after the time is up?

`match_rule` skips rules where `rule_expired()` is true, so the older deny underneath matches again. It's enforced in the data path with no job required.

What about a tunnel already open when time runs out or access is revoked?

`tunnel()` wakes every second, stops at the grant's deadline, and re-runs `match_rule` for that host and user, so it closes within about a second.

How do you stop a user approving their own request?

Approve, reject and revoke routes require the admin role, enforced server-side (403 otherwise). Hiding a button isn't security.

What is CSRF and how do you prevent it?

A malicious site makes your logged-in browser submit a form to the dashboard. Each session has a random token every POST must include, checked with `hmac.compare_digest`.

How are passwords stored?

Salted PBKDF2-SHA256 hashes via Werkzeug. (Scrypt, the default, isn't available in this Python build, so PBKDF2 is set explicitly.)

Why can't you scan each recv() chunk for keywords?

TCP has no message boundaries, so a keyword can be split across chunks. `StreamInspector` buffers into lines and holds back any possible keyword prefix.

Block, redact and log: what's the difference?

Block refuses or terminates, redact replaces the text and continues, log only records. A block wins over everything.

What is mutual TLS and why use it?

Both sides present certificates. The client certificate's Common Name becomes the identity, which can't be faked by editing a header. No certificate means `anonymous`.

How do rule changes apply without restarting?

Policy objects are immutable; changes build a new version and swap it atomically under a lock. A watcher polls the file mtime. After each swap, `revalidate()` kills open connections that are now denied.

Why does a blocked HTTPS site show "can't be reached" instead of a message?

The browser asks for a tunnel before TLS; when refused it discards the body. The block page works by answering 200 and doing TLS with a certificate from a locally trusted CA, only for blocked hosts.

Isn't trusting your own CA dangerous?

Yes, while trusted, whoever has `ca.key` can mint certificates for any site on that machine. Mitigations: dedicated CA, key locked down and git-ignored, 30-day certificates, strict host validation, only blocked hosts intercepted, trust removed afterwards.

How does the Browse page avoid SSRF?

Only http/https, the host must resolve to a public IP (so localhost and the LAN are refused), redirects are followed manually and re-checked at each hop, the preview is sandboxed with scripts off, and login is required.

What if the user turns off the proxy setting?

Filtering is bypassed. A proxy only filters traffic sent to it. Real enforcement needs network-level controls.

How would you scale it?

Replace JSON with a database or cache, use async I/O or a process pool, central logging, hashed credentials or SSO.

Why simulated services?

A safe, self-contained lab: reproducible demos and tests without touching real sites.

The full bank of 54 questions is in **STUDY_GUIDE** (md/pdf).

---

11. Live demo script (3 minutes)

Before you go on stage: open four terminals in the project folder with the virtual environment active (`source venv/bin/activate`). Check the demo works once, end to end.

```
python3 services.py
python3 proxy.py
python3 forward_proxy.py --block-page
python3 app.py
```

Trust the block-page CA once (the command is printed on first start) and set the macOS web proxies to `127.0.0.1:8080`. See `README_ACCESSWINDOW.md`. Accounts: `admin/admin-demo`, `bob/bob-demo`, `carol/carol-demo`.

1. Admin → **Website access** → add `youtube.com`, **Block**, all users.
2. Browser → youtube.com as bob → the **block page** (why, who, request button).
   Terminal fallback: `curl -x http://127.0.0.1:8080 --proxy-user bob:bob-demo https://youtube.com/ -I` → `403`.
3. Click **Request temporary access** → sign in as bob → 5 minutes + reason.
4. Admin → **Access requests → Approve**. Reload YouTube → it opens. Show **Temporary grants** counting

down.

5. **Revoke** (or wait) → the open connection is cut within a second and the block page returns. Show the **Activity log**.

6. Close with `packet_flow.html`: packet → auth → target → first-match rule → verdict → tunnel or 403.

> AFTER THE DEMO Remove the CA trust:
> `security delete-certificate -c "AccessWindow Block Page CA" ~/Library/Keychains/login.keychain-db`
> and turn the macOS web proxies off, or your Mac keeps routing through a proxy that is no longer running.

Cheat sheet: numbers and names

Dashboard	`127.0.0.1:5000`	Forward proxy	`127.0.0.1:8080`
TLS proxy / admin console	`:8443` / `127.0.0.1:9443`	Services	web 9001 · chat 9002 · echo 9003 · files 9004 · bulk 9005
Policy files	`config/policy.json` (A), `config/real_website_policy.json` (B)	Logs	`logs/access.log` (A), `logs/real_proxy_access.jsonl` (B)
Durations	5 / 15 / 30 / 60 minutes	Tunnel re-check	every 1 s
Related domains	`youtube.com` ⇒ `googlevideo.com`, `ytimg.com`, `youtu.be`, `youtube-nocookie.com`		

---

12. Flashcards: test yourself

1. One-sentence summary?

An ordered, first-match-wins rule list is evaluated for every connection; the dashboard edits that list; temporary access is just a rule with an expiry time.

2. Why is a temporary allow inserted at index 0?

First match wins. If appended, the older deny would match first and the grant would never apply.

3. Where is expiry enforced?

In the proxy's data path: rule_expired() compares expires_at with the clock on every request. No background job is needed for correctness.

4. What does the proxy see for an HTTPS site?

Only the hostname from CONNECT host:443. The path and content are encrypted.

5. Default action in Layer A vs Layer B?

A: deny (whitelist). B: allow (blacklist).

6. Two proxies: what's the difference?

A (netproxy): TLS proxy for simulated services with identity, rules and content filters. B (forward_proxy.py): real domains, hostname only.

7. How is user identity established in Layer A / B / dashboard?

A: client certificate Common Name (mTLS). B: Basic proxy auth (407 if wrong). Dashboard: login form + server-side session.

8. What stops 'notyoutube.com' matching 'youtube.com'?

host == domain or host.endswith('.' + domain).

9. Why DOMAIN_FAMILIES?

YouTube video comes from googlevideo.com, thumbnails from ytimg.com. Blocking only youtube.com would let a playing video continue.

10. What does revalidate() do?

After a policy swap it re-evaluates all open connections and kills the ones now denied. That makes filtering dynamic.

11. Why is Policy immutable?

So a thread can never see a half-applied rule list. Changes build a new version and swap atomically under a lock.

12. Why StreamInspector?

TCP has no message boundaries. A keyword can be split across packets, so it buffers into lines and holds back possible keyword prefixes.

13. Block vs redact vs log?

Block refuses or terminates; redact replaces text and continues; log only records. Block wins outright.

14. Why can't a browser show a proxy's block message for HTTPS?

It sends CONNECT before TLS and ignores the body of a refusal. Showing a page needs TLS interception with a trusted CA.

15. Biggest risk of the block page, and mitigation?

A trusted CA key could mint certs for any site. Dedicated CA, key chmod 600 + git-ignored, 30-day certs, strict host names, only blocked hosts, remove trust after the demo.

16. How is CSRF prevented?

Per-session random token required on every POST, compared with hmac.compare_digest.

17. How is SSRF prevented on Browse?

http/https only, host must resolve to a public IP, redirects re-checked each hop, sandboxed preview, login required.

18. Top 3 limitations?

Hostname-only for HTTPS; bypassable by turning the proxy off / VPN; demo-grade auth and JSON file storage.

19. Test status?

86 tests, 85 pass. test_tls_and_identity expects TLS 1.3 but system Python negotiates 1.2: an environment difference.